

Guía de ramas y despliegue — Para Eneko y Ekaitz

Fecha: 10 de abril de 2026 **Versión:** 1.0 **Para quién:** Eneko y Ekaitz (M&T Consulting)

¿Qué problema resuelve esto?

Ahora mismo tenéis UNA sola versión del CRM: `main`. Cuando hacéis un cambio, va directamente a producción — los clientes lo ven al instante. Si el cambio tiene un bug, los clientes lo sufren.

Es como hacer obras en un restaurante mientras los clientes están cenando.

Lo que vamos a montar es un sistema con **3 versiones paralelas**:

```
Tu ordenador (local) → develop (taller) → staging (ensayo) → main (restaurante)
```

Las 3 ramas explicadas

`main` = El restaurante abierto al público

- Es lo que ven los clientes AHORA MISMO en producción
- Solo llega código que ya está probado y funciona
- **NUNCA se trabaja directamente aquí**
- URL: la URL actual del CRM (la que usan los clientes)

`develop` = El taller donde montáis cosas nuevas

- Aquí es donde trabajáis Eneko y Ekaitz en el día a día
- Podéis romper cosas, probar ideas, meter la pata — sin que ningún cliente se entere
- Tiene su propia URL de Vercel para verlo en el navegador
- Tiene su propia base de datos (staging) para no tocar datos reales de clientes
- URL ejemplo: `develop-myt-crm.vercel.app`

`staging` = El ensayo general antes de abrir

- Cuando algo está listo en `develop`, lo pasáis a `staging`
- Es la última prueba antes de producción
- Si funciona aquí, se pasa a `main` con confianza
- URL ejemplo: `staging-myt-crm.vercel.app`

¿Cómo trabajaríais en el día a día?

Paso 1: Empezar a trabajar en algo nuevo

Abres la terminal en la carpeta del proyecto y escribes:

```
git checkout develop          # Te mueves a la rama de desarrollo
git pull                      # Traes los últimos cambios (por si Ekaitz ha subido algo)
git checkout -b feat/utm-tracking # Creas TU propia rama para tu tarea
```

Esto es como decir: "Voy a trabajar en el tracking de UTMs, me hago mi propia copia para no molestar a Ekaitz".

La rama se llama `feat/utm-tracking` — el nombre puede ser lo que quieras, pero la convención es:

- `feat/nombre` → para funcionalidades nuevas
- `fix/nombre` → para arreglar bugs
- `docs/nombre` → para documentación

Paso 2: Trabajar y subir cambios

Después de hacer cambios en el código:

```
git add src/App.jsx src/LeadDetail.jsx    # Añadir los archivos que has cambiado
git commit -m "feat: añadir campos UTM en LeadDetail"  # Guardar con un mensaje descriptivo
git push                                           # Subir a GitHub
```

Lo importante: Al hacer `push`, **Vercel automáticamente crea una URL de preview** solo para tu rama. Puedes copiar esa URL y mandársela a Ekaitz por WhatsApp para que la vea en el navegador sin instalar nada.

Paso 3: Fusionar tu trabajo (Pull Request)

Cuando has terminado tu tarea y funciona bien, creas un "Pull Request" (PR). Un PR es básicamente decir: *"He terminado esto, revisadlo y metedlo en develop"*.

Se puede hacer desde la web de GitHub (github.com → tu repositorio → Pull Requests → New) o desde terminal:

```
gh pr create --base develop --title "feat: UTM tracking en leads"
```

Ekaitz (o tú mismo) lo revisa, y si está bien, se aprueba y se fusiona haciendo clic en "Merge" en GitHub.

Paso 4: De develop a staging

Cuando hay suficientes cosas listas y queréis probarlas juntas antes de lanzar:

```
git checkout staging    # Te mueves a staging
git pull               # Traes lo último
git merge develop      # Traes todo lo de develop a staging
git push              # Vercel despliega staging automáticamente
```

Ahora podéis abrir la URL de staging en el navegador y probar todo junto. Si algo falla, se arregla en develop y se vuelve a fusionar.

Paso 5: De staging a producción (main)

Cuando todo está probado en staging y funciona:

```
git checkout main      # Te mueves a producción
git pull              # Traes lo último
git merge staging      # Pasas todo a producción
git push             # Vercel despliega a producción automáticamente
```

Los clientes ven los cambios en unos minutos.

¿Y si trabajáis los dos a la vez?

Cada uno crea su propia rama desde `develop` :

- Eneko trabaja en `feat/utm-tracking`
- Ekaitz trabaja en `feat/motivo-perdida`

Trabajáis en paralelo, cada uno en su rama. Cuando terminéis, cada uno hace un PR a `develop` .

Si habéis tocado archivos diferentes: se fusiona automáticamente, sin problemas.

Si habéis tocado el mismo archivo: Git os avisa con un "conflicto". No pasa nada — simplemente hay que elegir qué versión queda (o combinar ambas). Esto se resuelve abriendo el archivo, viendo las dos versiones marcadas, y eligiendo cuál es la correcta.

Vercel: cómo funciona con las ramas

Vercel está conectado a vuestro repositorio de GitHub. Cada vez que hacéis push a cualquier rama, Vercel lo detecta automáticamente y despliega:

Rama	Qué hace Vercel	URL resultado
main	Deploy a producción	tu-dominio.com (la URL real del CRM)
staging	Deploy a preview fija	staging--myt-crm.vercel.app
develop	Deploy a preview fija	develop--myt-crm.vercel.app
feat/lo-que-sea	Deploy a preview temporal	URL temporal que Vercel genera

No hay que hacer nada especial: solo con hacer `git push`, Vercel se encarga. Podéis ver el deploy en el dashboard de Vercel o en la pestaña "Deployments".

Configuración necesaria en Vercel (se hace una vez)

1. Ir a **Vercel Dashboard** → **Settings** → **Git**
2. **Production Branch:** `main` (ya debería estar así)
3. Las preview URLs se generan automáticamente para todas las demás ramas

Variables de entorno por entorno

En **Vercel Dashboard** → **Settings** → **Environment Variables**:

Variable	Production	Preview
VITE_SUPABASE_URL	URL de Supabase producción	URL de Supabase staging
VITE_SUPABASE_ANON_KEY	Key de producción	Key de staging

Esto hace que cuando Vercel despliega `main`, use la base de datos de producción, y cuando despliega cualquier otra rama, use la base de datos de staging. **Los datos de los clientes están siempre protegidos.**

Supabase: cómo funcionan las "ramas" de base de datos

Opción A: Supabase Branching (automático)

Supabase tiene una funcionalidad llamada **Branching** que crea copias temporales de la base de datos automáticamente. Está disponible en el plan Pro (que ya tenéis contratado).

Cómo funciona:

1. Habilitáis Branching en **Supabase Dashboard** → **Settings** → **Branching** → **Enable**
2. Las migraciones SQL (los cambios en la estructura de la BD) las ponéis en la carpeta `supabase/migrations/` del repo
3. Cuando hacéis un PR en GitHub, Supabase crea automáticamente una copia de la BD con esas migraciones aplicadas
4. Cuando el PR se fusiona a `main`, las migraciones se aplican a producción automáticamente

Opcion B: Proyecto staging separado (más simple)

Si el Branching automático os parece complejo al principio:

1. Crear un **segundo proyecto de Supabase** (gratis dentro del plan Pro)

2. Replicar la estructura de tablas de producción
3. Usar esa BD para todo lo que no sea `main`
4. Las migraciones se aplican primero en staging (a mano) y después en producción

Recomendación: Empezar con la Opción B (más simple, más control) y migrar a la Opción A cuando estéis cómodos.

n8n: el caso especial

n8n **NO tiene sistema de ramas**. Los workflows viven en un servidor compartido de producción. Esto es lo que hay que tener claro:

Para cambios pequeños (ej: añadir un campo a un insert)

- Hacedlo directamente en la UI de n8n
- Probad con un lead de test antes de dar por bueno
- No necesita rama ni nada

Para cambios grandes (ej: nuevo workflow completo)

1. Crear el workflow nuevo con un **webhook path de test** (ej: `/webhook-test/captura-lead`)
2. Apuntar vuestro entorno de staging a ese webhook
3. Probar hasta que funcione
4. Cambiar el path al de producción

Para tocar WF2 (el chatbot de WhatsApp, el workflow más grande y crítico)

1. **SIEMPRE** duplicar WF2 primero (hay un backup: `6J7at0lEbuniU9Ma`)
 2. Trabajar en la copia
 3. Cuando esté validado, hacer los mismos cambios en el WF2 original
 4. Verificar que sigue teniendo 47 nodos (no se ha perdido nada)
-

Comandos rápidos de referencia

Lo más común

```
# Ver en qué rama estás
git branch

# Ver qué archivos has cambiado
git status

# Cambiar de rama
git checkout nombre-rama

# Crear rama nueva desde la actual
git checkout -b feat/nombre-de-la-tarea

# Guardar cambios
git add archivo1.jsx archivo2.jsx
git commit -m "feat: descripción corta del cambio"

# Subir a GitHub
git push

# Si es la primera vez que subes esta rama:
git push -u origin nombre-rama
```

```
# Traer los últimos cambios de GitHub
git pull
```

Flujo completo típico

```
# 1. Empezar tarea
git checkout develop
git pull
git checkout -b feat/mi-tarea

# 2. Trabajar... hacer cambios... probar en local...

# 3. Guardar y subir
git add .
git commit -m "feat: lo que he hecho"
git push -u origin feat/mi-tarea

# 4. Ir a GitHub → crear Pull Request → base: develop

# 5. Cuando está aprobado → Merge en GitHub

# 6. Cuando hay suficientes cosas listas para staging:
git checkout staging
git pull
git merge develop
git push

# 7. Probar en staging... si todo OK:
git checkout main
git pull
git merge staging
git push
# ¡En producción!
```

Con GitHub Desktop

Si usáis GitHub Desktop (que ya lo usáis):

1. **Cambiar rama:** Menú desplegable arriba "Current Branch" → seleccionar rama
2. **Crear rama:** "Current Branch" → "New Branch" → nombre → crear desde `develop`
3. **Commit:** Panel izquierdo muestra archivos cambiados → escribir mensaje → "Commit to [rama]"
4. **Push:** Botón "Push origin" arriba
5. **Pull Request:** Botón "Create Pull Request" → se abre GitHub en el navegador
6. **Merge:** En la web de GitHub → botón "Merge pull request"

Reglas de oro

1. **Nunca trabajéis directamente en `main`** . Siempre desde una rama de feature que sale de `develop` .
2. **Antes de empezar a trabajar, siempre `git pull`** para traer lo último.
3. **Un commit por tarea lógica**, no un megacommit con todo mezclado.
4. **Los mensajes de commit en español están bien**. Formato: `feat: descripción` o `fix: descripción` .
5. **Si algo se rompe en producción:** no entrar en pánico. Se puede revertir el último deploy desde el dashboard de Vercel en un clic.
6. **Si tenéis un conflicto de merge:** no borréis nada a lo loco. Preguntad o resolvedlo juntos.

Glosario rápido

Término	Qué significa
Rama (branch)	Una copia paralela del código donde trabajas sin afectar a los demás
Commit	Un "punto de guardado" con un mensaje que describe qué has cambiado
Push	Subir tus commits a GitHub (y que Vercel lo despliegue)
Pull	Traer los cambios que otros han subido a GitHub
Pull Request (PR)	Petición para fusionar tu rama en otra (ej: tu feature en develop)
Merge	Fusionar una rama en otra (combinar los cambios)
Conflicto	Cuando dos personas han cambiado la misma línea del mismo archivo
Preview URL	URL temporal que Vercel crea para ver tu rama en el navegador
Deploy	Poner código en un servidor donde se puede acceder vía URL
Migration	Cambio en la estructura de la base de datos (añadir columna, tabla, etc.)